De Casteljau Algorithm

Quadratic Bézier — Plots.jl Visualisation

Code and Plot Explanation for Julia Beginners

`12-de-casteljau-plots.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains `12-de-casteljau-plots.jl`. The script visualises the **De Casteljau algorithm** — a geometric method for evaluating Bézier curves — applied to the same quadratic curve as script 11. New concepts: linear interpolation (`lerp`), multi-level geometric construction, named tuple return values, and plotting multiple coloured snapshots of the algorithm.

Contents

1 The Plot **3**

2 The De Casteljau Algorithm — What Is It? **3**

3 Reading the Plot **4**
3.1 Background elements 4
3.2 The three coloured snapshots 4
3.3 What the construction lines reveal 5

4 Type Map **5**

5 Data and Constants **5**

6 The `DeCasteljauQuadratic` Struct **6**

7 The Math Functions **6**
7.1 `lerp` — linear interpolation 6

7.2 `steps` — one full De Casteljau construction 6
7.3 `dc_curve` — the full curve via De Casteljau 8

8 `draw_and_save` — Building the Plot 9
8.1 Base figure and static layers 9
8.2 The snapshot loop 9
8.3 Level-1 segment and dots 9
8.4 The level-2 diamond 10
8.5 Snapshot labels 10

9 Entry Point 11

1. The Plot

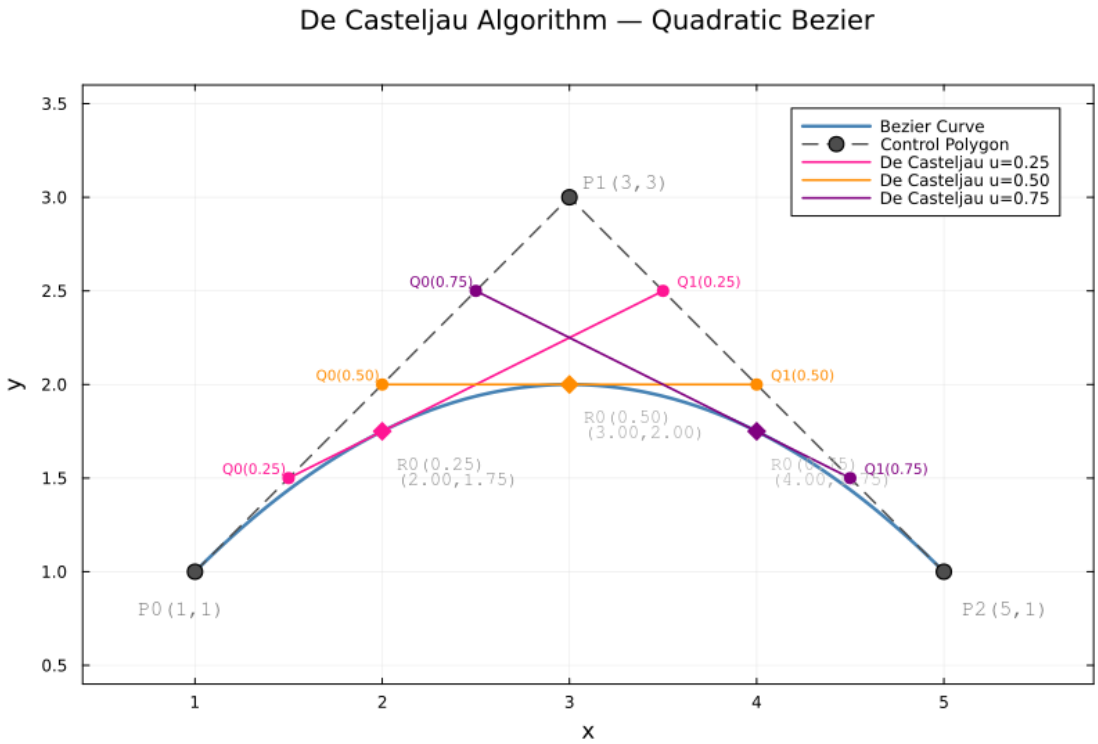


Figure 1: De Casteljau algorithm at $u = 0.25$ (pink), $u = 0.50$ (orange), $u = 0.75$ (purple).

2. The De Casteljau Algorithm — What Is It?

Script 11 evaluated the Bézier curve using the Bernstein polynomial formula. This script shows a completely different approach that produces the exact same curve, but geometrically.

The **De Casteljau algorithm** evaluates a Bézier curve at parameter u by repeated linear interpolation. For a quadratic curve with control points P_0, P_1, P_2:

Level 0 (given): the three control points.

Level 1 (interpolate): compute two new points by interpolating between adjacent control points at ratio u:

$$Q_0 = (1-u)P_0 + uP_1 \qquad Q_1 = (1-u)P_1 + uP_2$$

Level 2 (interpolate again): compute one final point by interpolating between Q_0 and Q_1 at ratio u:

$$R_0 = (1-u)Q_0 + uQ_1$$

R_0 is the point on the Bézier curve at parameter u. Repeating this for all $u \in [0, 1]$ traces the full curve.

> **Why is this useful?** De Casteljau is numerically stable (no polynomial evaluation, no large powers), easy to implement recursively for any degree, and forms the basis of curve subdivision algorithms used in font rendering and CAD software. The Bernstein formula and De Casteljau always give the same curve point — they are two ways of computing the same thing.

3. Reading the Plot

3.1. Background elements

Blue curve — the full Bézier curve, same as script 11.

Gray dashed polygon with black dots — the control polygon connecting $P_0(1, 1)$, $P_1(3, 3)$, $P_2(5, 1)$.

3.2. The three coloured snapshots

The algorithm is shown at three values of u, each in a different colour. For each snapshot:

- A **solid coloured line** connects Q_0 and Q_1 (the level-1 intermediate points). This line is the "level-1 construction segment."
- Two **small filled circles** mark Q_0 and Q_1 with labels `Q0(u)` and `Q1(u)`.
- A **diamond** marks R_0 — the final curve point at that u value. The diamond sits exactly on the blue curve.
- A **label** below the diamond shows `R0(u) / (x, y)`.

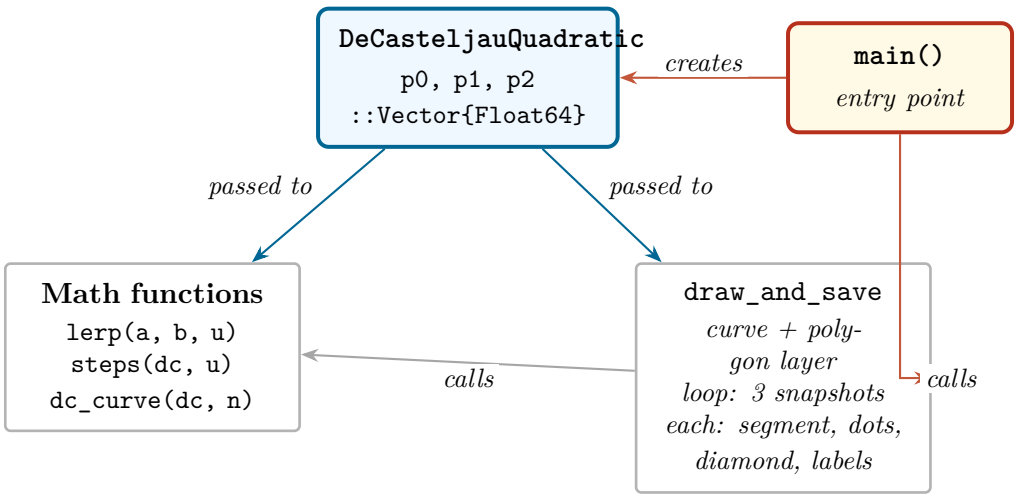
> **Reading the $u = 0.50$ snapshot (orange):**
>
> $Q_0(0.50)$ is the midpoint of P_0P_1: $(1 + 3)/2 = 2,\ (1 + 3)/2 = 2 \to (2, 2)$.
>
> $Q_1(0.50)$ is the midpoint of P_1P_2: $(3 + 5)/2 = 4,\ (3 + 1)/2 = 2 \to (4, 2)$.
>
> $R_0(0.50)$ is the midpoint of Q_0Q_1: $(2 + 4)/2 = 3,\ (2 + 2)/2 = 2 \to (3, 2)$.
>
> This matches the orange diamond at $(3.00, 2.00)$ on the plot. ✓
>
> The orange construction segment is the horizontal line from $(2, 2)$ to $(4, 2)$.

> **Reading the $u = 0.25$ snapshot (pink):**
>
> $Q_0(0.25) = 0.75 \cdot P_0 + 0.25 \cdot P_1 = 0.75(1, 1) + 0.25(3, 3) = (1.5, 1.5)$.
>
> $Q_1(0.25) = 0.75 \cdot P_1 + 0.25 \cdot P_2 = 0.75(3, 3) + 0.25(5, 1) = (3.5, 2.5)$.
>
> $R_0(0.25) = 0.75 \cdot Q_0 + 0.25 \cdot Q_1 = 0.75(1.5, 1.5) + 0.25(3.5, 2.5) = (2.0, 1.75)$.
>
> The pink diamond sits at $(2.00, 1.75)$. ✓

3.3. What the construction lines reveal

The coloured construction segments Q_0Q_1 visually show how the curve point is "found" geometrically: the curve point R_0 always lies on its construction segment, at fraction u along that segment. The three crossing construction lines in the plot illustrate why the algorithm is elegant — each line narrows in on the curve from a different angle.

4. Type Map



5. Data and Constants

```
const CONTROL_POINTS = [
    1.0  1.0;   # P0
    3.0  3.0;   # P1
    5.0  1.0;   # P2
]

const U_SNAPSHOTS     = [0.25, 0.50, 0.75]
const SNAPSHOT_COLORS = [:deeppink, :darkorange, :purple]
```

`CONTROL_POINTS` is identical to script 11.

`U_SNAPSHOTS` lists the three u values at which the De Casteljau construction is drawn. `SNAPSHOT_COLORS` is a `Vector{Symbol}` of colour names — one per snapshot. The two vectors are paired: $u = 0.25$ uses `:deeppink`, $u = 0.50$ uses `:darkorange`, $u = 0.75$ uses `:purple`.

These pairs are iterated together with `zip` in the drawing loop, which is why the pink lines correspond to $u = 0.25$ and so on.

6. The DeCasteljauQuadratic Struct

```
struct DeCasteljauQuadratic
    p0::Vector{Float64}
    p1::Vector{Float64}
    p2::Vector{Float64}
end

DeCasteljauQuadratic(pts::Matrix{Float64}) =
    DeCasteljauQuadratic(pts[1,:], pts[2,:], pts[3,:])
```

Identical in structure to `BezierQuadratic` from script 11 — same three vector fields, same row-slice constructor. The name change from `BezierQuadratic` to `DeCasteljauQuadratic` signals the different algorithm used to evaluate it.

7. The Math Functions

7.1. lerp — linear interpolation

```
lerp(a, b, u) = (1 - u) .* a .+ u .* b
```

`lerp` stands for **linear interpolation**. Given two points `a` and `b` and a parameter `u` $\in [0, 1]$, it returns the point that is fraction u of the way from `a` to `b`:

$$\text{lerp}(a, b, u) = (1 - u)\, a + u\, b$$

At $u = 0$ the result is exactly `a`. At $u = 1$ the result is exactly `b`. At $u = 0.5$ the result is the midpoint.

The broadcast operators `.*` and `.+` make this work for both scalar arguments and 2-D point vectors without any change to the formula.

> **Scalar lerp:** `lerp(1.0, 5.0, 0.25)` $= 0.75 \times 1.0 + 0.25 \times 5.0 = 2.0$
>
> **Vector lerp:** `lerp([1.0, 1.0], [3.0, 3.0], 0.25)` $= 0.75 \times [1, 1] + 0.25 \times [3, 3] = [0.75, 0.75] + [0.75, 0.75] = [1.5, 1.5]$
>
> This is Q_0 at $u = 0.25$ — the point $\frac{1}{4}$ of the way from P_0 to P_1.

7.2. steps — one full De Casteljau construction

```
function steps(dc::DeCasteljauQuadratic, u::Float64)
    q0 = lerp(dc.p0, dc.p1, u)
    q1 = lerp(dc.p1, dc.p2, u)
    r0 = lerp(q0, q1, u)
    return (
        level0 = vcat(dc.p0', dc.p1', dc.p2'),
        level1 = vcat(q0',    q1'),
        level2 = r0,
```

```
    )
end
```

The computation: Three `lerp` calls implement the two levels of De Casteljau interpolation. `q0` and `q1` are the level-1 points; `r0` is the final curve point.

Named tuple return: The function returns a **named tuple** — a tuple where each element has a name. In Julia, `(level0 = ..., level1 = ..., level2 = ...)` creates a named tuple. The caller accesses fields by name: `s.level0`, `s.level1`, `s.level2`.

This is more readable than returning a plain tuple `(matrix1, matrix2, vector)` and having to remember which position is which.

The data in each level:

`level0` is the 3×2 control point matrix (same shape as `cp` in script 11). Built by stacking three transposed row vectors with `vcat`.

`level1` is a 2×2 matrix holding Q_0 and Q_1 as rows. Built by stacking two transposed row vectors.

`level2` is a length-2 vector holding $R_0 = (x, y)$.

Full `steps(dc, 0.25)` output:

$$\texttt{level0} = \begin{bmatrix} 1.0 & 1.0 \\ 3.0 & 3.0 \\ 5.0 & 1.0 \end{bmatrix} \quad \text{(always the same — control points)}$$

$Q_0 = \text{lerp}(P_0, P_1, 0.25) = (1.5, 1.5)$

$Q_1 = \text{lerp}(P_1, P_2, 0.25) = (3.5, 2.5)$

$$\texttt{level1} = \begin{bmatrix} 1.5 & 1.5 \\ 3.5 & 2.5 \end{bmatrix}$$

$R_0 = \text{lerp}(Q_0, Q_1, 0.25) = (2.0, 1.75)$

$$\texttt{level2} = [2.0,\ 1.75]$$

Full `steps(dc, 0.50)` output:

$Q_0 = \text{lerp}(P_0, P_1, 0.50) = (2.0, 2.0)$

$Q_1 = \text{lerp}(P_1, P_2, 0.50) = (4.0, 2.0)$

$$\texttt{level1} = \begin{bmatrix} 2.0 & 2.0 \\ 4.0 & 2.0 \end{bmatrix}$$

$R_0 = \text{lerp}(Q_0, Q_1, 0.50) = (3.0, 2.0)$

$$\texttt{level2} = [3.0,\ 2.0]$$

This explains the flat orange construction segment (both Q points have $y = 2$) and the diamond exactly at the peak of the curve.

Full `steps(dc, 0.75)` output:

$Q_0 = \text{lerp}(P_0, P_1, 0.75) = (2.5, 2.5)$

$Q_1 = \text{lerp}(P_1, P_2, 0.75) = (4.5, 1.5)$

$$\texttt{level1} = \begin{bmatrix} 2.5 & 2.5 \\ 4.5 & 1.5 \end{bmatrix}$$

$R_0 = \text{lerp}(Q_0, Q_1, 0.75) = (4.0, 1.75)$

$$\texttt{level2} = [4.0,\ 1.75]$$

7.3. `dc_curve` — the full curve via De Casteljau

```julia
function dc_curve(dc::DeCasteljauQuadratic, n::Int=200)
    u_vals = LinRange(0.0, 1.0, n)
    hcat([steps(dc, u).level2 for u in u_vals]...)'
end
```

For each of 200 u values, calls `steps(dc, u)` and extracts only `.level2` — the final curve point. The `.level2` after the function call is **field access on the return value**: it reads the `level2` field of the named tuple returned by `steps`, without storing the tuple in a variable first.

The rest of the function (`hcat`, splat, transpose) is identical to `curve()` in script 11 — same pattern, same result shape (200×2).

Content of `cpts = dc_curve(dc)` (first and last few rows):

Row	x	y
1	1.0000	1.0000
2	1.0402	1.0793
⋮	⋮	⋮
100	3.0000	2.0000
⋮	⋮	⋮
200	5.0000	1.0000

This is the same curve as script 11 — the De Casteljau algorithm and Bernstein formula always produce the same curve points. Only the computation path differs.

8. `draw_and_save` — Building the Plot

8.1. Base figure and static layers

The `plot()` call, the curve layer (`plot!(plt, cpts...)`), and the control polygon layer are identical in structure to script 11. One difference: the control polygon here uses `COLOR_POLYGON = :gray30` (dark grey) instead of crimson, giving the background skeleton a more neutral look so the coloured snapshots stand out.

The control polygon is extracted via `steps(dc, 0.0).level0` rather than constructing it with `vcat(bz.p0', bz.p1', bz.p2')` directly. At $u = 0$, `level0` always returns the full control point matrix — calling `steps` is consistent with the rest of the code even if slightly roundabout.

8.2. The snapshot loop

```
for (u, color) in zip(u_snapshots, colors)
    s  = steps(dc, u)
    l1 = s.level1
    r0 = s.level2
    ...
end
```

`zip(u_snapshots, colors)` pairs each u value with its colour: `(0.25, :deeppink)`, `(0.50, :darkorange)`, `(0.75, :purple)`. Each iteration draws one full De Casteljau snapshot.

`s = steps(dc, u)` calls the construction function once and stores the full result. `l1 = s.level1` and `r0 = s.level2` extract the two fields needed for plotting.

8.3. Level-1 segment and dots

```
# Level-1 Q0-Q1 segment
plot!(plt, l1[:,1], l1[:,2],
    color     = color,
    linewidth = 1.6,
    label     = @sprintf("De Casteljau u=%.2f", u),
)

# Level-1 Q0, Q1 dots
scatter!(plt, l1[:,1], l1[:,2],
    color             = color,
    markersize        = 5,
    markerstrokewidth = 0,
    label             = false,
)
```

`l1[:,1]` and `l1[:,2]` extract the x and y columns from the 2×2 level-1 matrix. `plot!` draws the construction segment; `scatter!` draws the two Q dots on top of it.

`label = false`: The `scatter!` call should not add a legend entry — the segment `plot!` already provides the legend entry for this snapshot. Passing `label = false`

suppresses the legend entry for that layer entirely.

8.4. The level-2 diamond

```
scatter!(plt,
    [r0[1]], [r0[2]],
    color             = color,
    marker            = :diamond,
    markersize        = 7,
    markerstrokewidth = 0,
    label             = false,
)
```

`r0` is a length-2 vector `[x, y]`. `scatter!` expects vectors of coordinates, not scalar values. `[r0[1]]` wraps the scalar x value in a length-1 vector; `[r0[2]]` does the same for y. This is necessary because `scatter!(plt, x, y)` requires both `x` and `y` to be arrays.

`marker = :diamond` selects the diamond shape — the rotated square visible on the plot at each R_0 point.

> **Why `[r0[1]]` and not just `r0[1]`?**
>
> `r0[1] = 2.0` (a scalar Float64). Passing a scalar to `scatter!` would cause a method error. `[2.0]` is a length-1 vector `Vector{Float64}`, which `scatter!` accepts.
>
> This pattern `[scalar]` — wrapping a single value in square brackets to make it a vector — is common in Julia plotting code whenever you need to plot a single isolated point.

8.5. Snapshot labels

```
# Q0, Q1 labels
for (j, (qname, (dx, dy))) in enumerate(zip(["Q0","Q1"], Q_OFFSETS))
    annotate!(plt, l1[j,1]+dx, l1[j,2]+dy,
        text(@sprintf("%s(%.2f)", qname, u), color, :left, 7))
end

# R0 label
annotate!(plt, r0[1]+0.08, r0[2]-0.22,
    text(@sprintf("R0(%.2f)\n(%.2f,%.2f)", u, r0[1], r0[2]),
         color, :left, 7, :bold))
```

The Q label loop uses `enumerate(zip(...))` to get both the index `j` (for accessing `l1[j,1]`) and the name/offset pair. `Q_OFFSETS = [(-0.35, 0.05), (0.08, 0.05)]` shifts Q_0 labels to the left (so they don't overlap the construction line) and Q_1 labels to the right.

The `R0` label includes both the u value and the (x, y) coordinates, formatted in bold with a line break.

All labels produced by the loop for $u = 0.25$ (pink):

Label	Text string	Position
Q_0	`"Q0(0.25)"`	$(1.5 - 0.35,\ 1.5 + 0.05) = (1.15, 1.55)$
Q_1	`"Q1(0.25)"`	$(3.5 + 0.08,\ 2.5 + 0.05) = (3.58, 2.55)$
R_0	`"R0(0.25)\n(2.00,1.75)"`	$(2.0 + 0.08,\ 1.75 - 0.22) = (2.08, 1.53)$

9. Entry Point

```julia
function main()
    dc = DeCasteljauQuadratic(CONTROL_POINTS)
    draw_and_save(dc, U_SNAPSHOTS, SNAPSHOT_COLORS,
                  "12-bezier-decasteljau-plots.png")
end

main()
```

`draw_and_save` now takes four arguments instead of three — the snapshot u values and colours are passed in rather than being hardcoded inside the function. This makes it easy to change the snapshots or colours without editing the drawing function itself.

How to run:

```julia
julia 12-de-casteljau-plots.jl
```

Saves `12-bezier-decasteljau-plots.png` and opens a window. Press Enter to close.